

MSCIT sem 3
Employment Management System Project
Mobile and web development
ID:202412061

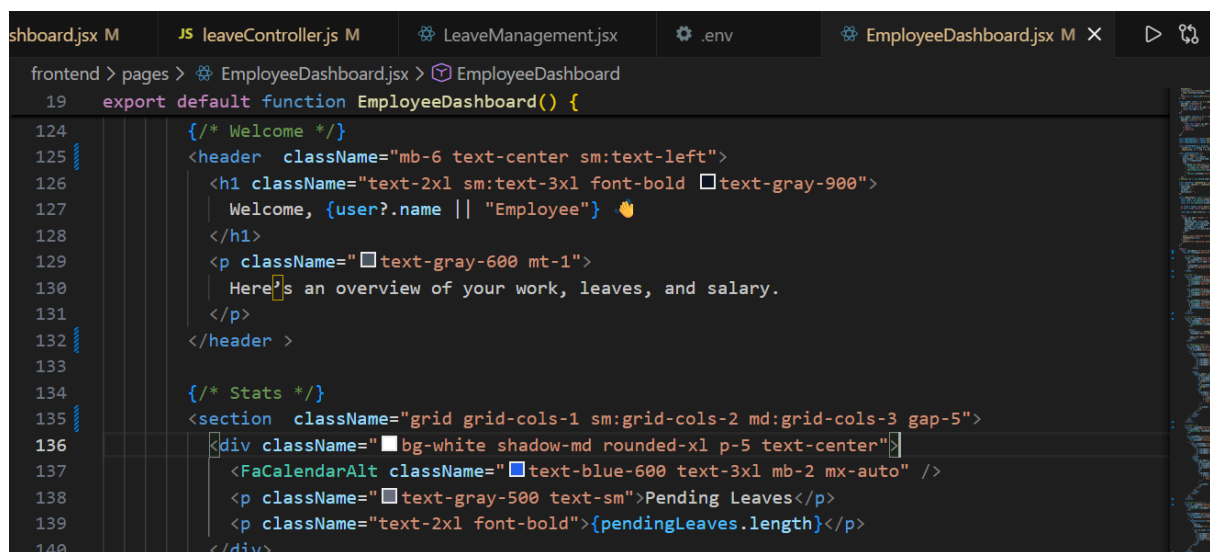
Name: Parmar Shenal Kamlesh

(POSTMAN screenshots available in report file)

=====

HTML, CSS and Javascript (5 Marks)

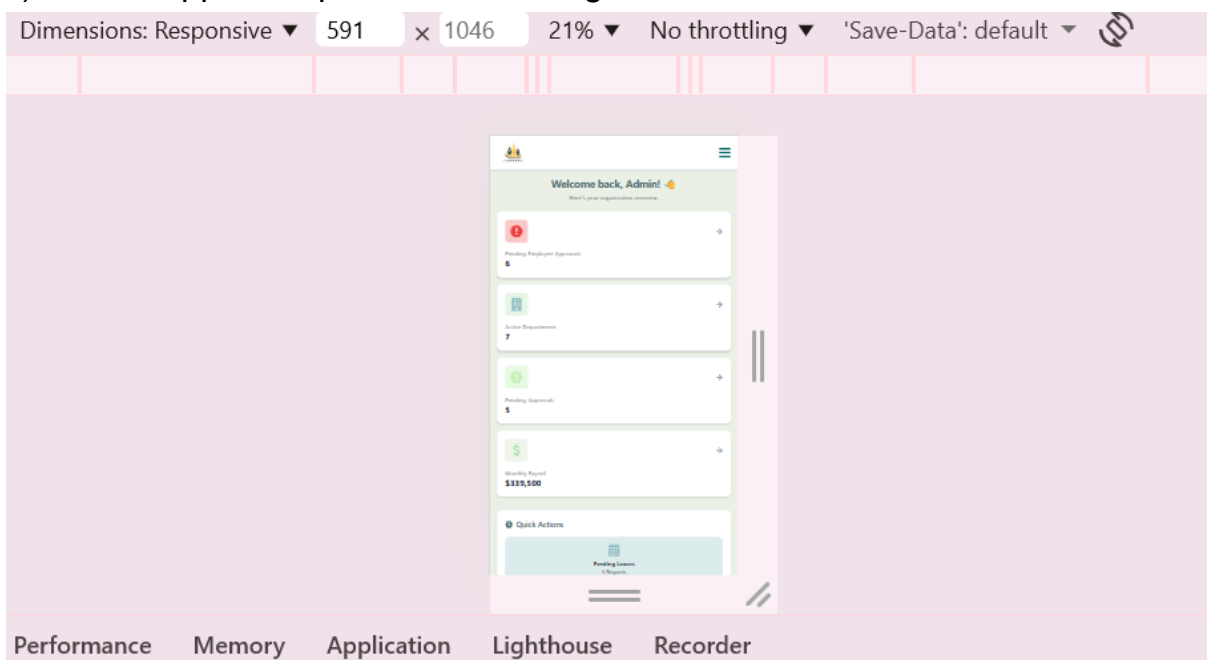
1)Use semantic elements: <header>, <Section> <nav> etc instead of <div> wherever applicable.

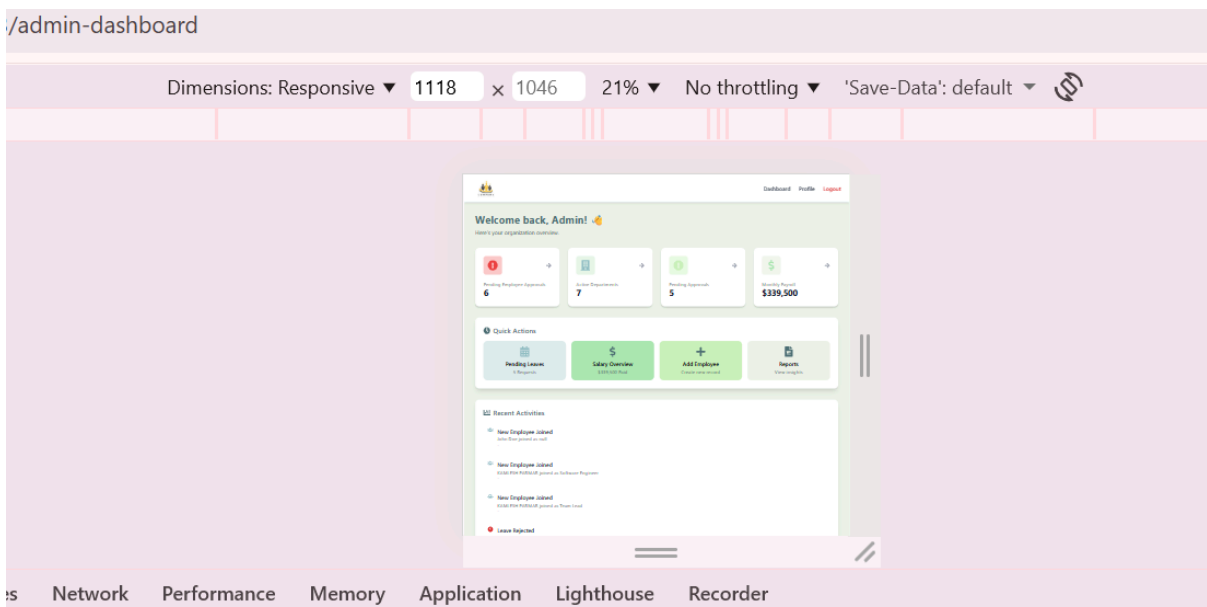
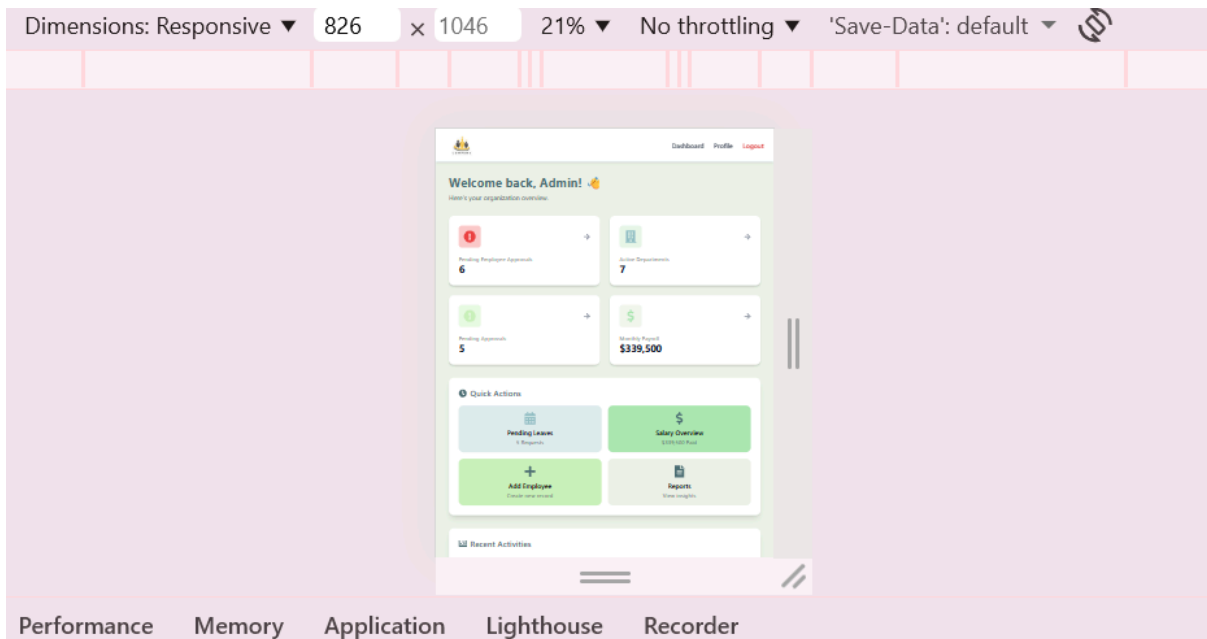


```
shboard.jsx M JS leaveController.js M LeaveManagement.jsx .env EmployeeDashboard.jsx M x
frontend > pages > EmployeeDashboard.jsx > EmployeeDashboard
19 export default function EmployeeDashboard() {
124   {/* Welcome */}
125   <header className="mb-6 text-center sm:text-left">
126     <h1 className="text-2xl sm:text-3xl font-bold text-gray-900">
127       Welcome, {user?.name || "Employee"} 🙌
128     </h1>
129     <p className="text-gray-600 mt-1">
130       Here's an overview of your work, leaves, and salary.
131     </p>
132   </header >
133
134   {/* Stats */}
135   <section className="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 gap-5">
136     <div className="bg-white shadow-md rounded-xl p-5 text-center">
137       <FaCalendarAlt className="text-blue-600 text-3xl mb-2 mx-auto" />
138       <p className="text-gray-500 text-sm">Pending Leaves</p>
139       <p className="text-2xl font-bold">{pendingLeaves.length}</p>
140     </div>
```

```
AdminDashboard.jsx M X
frontend > pages > AdminDashboard.jsx > AdminDashboard
26 export default function AdminDashboard() {
200   return (
201     <main className="p-4 sm:p-6 md:p-8 space-y-8 bg-[#ECF4E8] min-h-screen">
202       <section className="max-w-7xl mx-auto">
203
204         {/* Welcome Section */}
205         <header className="mb-6 sm:mb-8 text-center sm:text-left">
206           <h1 className="text-2xl sm:text-3xl font-bold text-[#4F6F75] mb-2">
207             Welcome back, {user?.full_name || "Admin"}! 🙌
208           </h1>
209           <p className="text-gray-600 text-sm sm:text-base">
210             Here's your organization overview.
211           </p>
212         </header>
213
214         {/* Stats Section */}
215         <section className="
216           grid
217           grid-cols-1
218           sm:grid-cols-2
219           lg:grid-cols-4
220           gap-4 sm:gap-6
221           mb-8
222         ">
223           {stats.map((stat, i) => (
224             <Link
225               to={stat.link}
226             >
```

2)should support responsive web design





3. Tailwind CSS should be used for styling-used everywhere this only

4. Use atleast 1 thrid party Web API in your project.:brevo for email

```
JS MailService.js X
backend > utils > JS MailService.js > [⌘] sendMail > [⌘] response > [⌘] headers
1  import axios from "axios";
2
3  export const sendMail = async ({ to, subject, text, html }) => {
4    try {
5      const payload = {
6        sender: { email: process.env.BREVO_SENDER },
7        to: [{ email: to }],
8        subject,
9        textContent: text,
10       htmlContent: html || `

${text}</p>`,
11     };
12
13     console.log("email payload:", payload);
14
15     const response = await axios.post(
16       "https://api.brevo.com/v3/smtp/email",
17       payload,
18       {
19         headers: {
20           "Content-Type": "application/json",
21           "api-key": process.env.BREVO_API_KEY,
22         },
23       }
24     );
25
26     console.log("Email sent:", response.data);
27   } catch (error) {
28     console.error("Email error:", error.message);
29     console.error(error.response?.data);
30   }
31 }


```

React:

1. Use function components preferably
2. Use fetch API to make HTTP requests to server side
3. Use atleast these hooks in your project: useState, useEffect, useRef and useContext -----(used in many files)

```
App.jsx M AuthContext.jsx M X
ontend > context > AuthContext.jsx > [🔍] userContext
2 import { useContext, useEffect } from "react";
3 import { useState } from "react";
4 import { createContext } from "react";
5 import Login from "../pages/Login";
6 const api = axios.create({
7   | baseUrl: `${import.meta.env.VITE_API_URL?.replace(/\$/ , "")}/api`,
8   | });
9 api.interceptors.request.use((config) => {
10   | const token = localStorage.getItem("token");
11   | // console.log("TOKEN SENDING → ", token);
12   | if (token) {
13   |   | config.headers.Authorization = `Bearer ${token}`;
14   |   | }
15   | return config;
16   | });
17 export const userContext = createContext();
18 const AuthContext = ({ children }) => {
19   | const [user, setUser] = useState();
20   | const [Loading, setLoading] = useState(true);
21   | console.log("authcontext called");
22   | useEffect(() => {
23   |   | const verifyToken = async () => {
24   |   |   | // console.log("token in verify b4:", token);
25   |   |   | const token = localStorage.getItem("token");
26   |   |   | // console.log("token in verify:", token);
27   |   |   | if (!token) {
28   |   |   |   | setUser(null);
29   |   |   |   | setLoading(false);
30   |   |   |   | return;
  
```

4. Use ReactDOM for navigation

frontend > src > main.jsx > ...

```
1  import { StrictMode } from "react";
2  import { createRoot } from "react-dom/client";
3  import "./index.css";
4  import App from "./App.jsx";
5  import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
6  import AuthContext from "../context/AuthContext.jsx";
7  import { BrowserRouter } from "react-router-dom";
8
9  const queryClient = new QueryClient();
10
11  createRoot(document.getElementById("root")).render(
12    <StrictMode>
13      <QueryClientProvider client={queryClient}>
14        <BrowserRouter>
15          <AuthContext>
16            <App />
17          </AuthContext>
18        </BrowserRouter>
19      </QueryClientProvider>
20    </StrictMode>
21  );
22
```

```

</RouteContainer />
<Routes>
  <Route path="/" element={<Login />}></Route>
  <Route path="/login" element={<Login />}></Route>
  <Route path="/reports" element={<Reports />}></Route>
  <Route
    path="/admin-dashboard"
    element={
      <ProtectedRoute role="admin">
        <AdminDashboard />
      </ProtectedRoute>
    }
  />

  <Route
    path="/employee-dashboard"
    element={
      <ProtectedRoute role="employee">
        <EmployeeDashboard />
      </ProtectedRoute>
    }
  />

  <Route
    path="/employeeManagement"
    element={<EmployeeManagement />}
  ></Route>

```

B. Node and ExpressJS Backend (25 Marks)

Use development dependencies: morgan nodemon cors

```

app.use(cors({
  origin: function (origin, callback) {
    if (!origin) return callback(null, true);
    if (allowedOrigins.includes(origin)) return callback(null, true);
    return callback(new Error(`CORS blocked for origin: ${origin}`), false);
  },
  credentials: true,
  methods: ["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
  allowedHeaders: ["Content-Type", "Authorization"]
}));

app.get("/test-email", async (req, res) => {
  await sendMail({
    to: "shenal5420@gmail.com",
    subject: "Test email",
    text: "Hello from Brevo!",
  });
  res.send("Email sent");
});

app.use(morgan("dev"));
app.use(express.json());

```

2. Structure URLs with clear endpoints (e.g., /api/users/:id).

```
App.jsx M    JS server.js M    JS leaveRoutes.js M X
backend > routes > JS leaveRoutes.js > ...
1  import express from "express";
2  import {
3      createLeave,
4      getLeaves,
5      getLeaveById,
6      getMyLeaves,
7      updateLeave,
8      deleteLeave,
9      updateLeaveStatus,
10 } from "../controller/leaveController.js";
11
12 const router = express.Router();
13
14 router.post("/", createLeave);
15 router.get("/", getLeaves);
16 router.get("/:id", getLeaveById);
17 router.get("/my-leaves/:id", getMyLeaves);
18 router.put("/:id", updateLeaveStatus);
19 router.patch("/:id", updateLeave);
20 router.delete("/:id", deleteLeave);
21
22 export default router;
23
```

3. Each API endpoint should represent a unique resource. Use route parameters to identify a resource

4. Implement proper status codes for responses (e.g., 200 for success, 404 for not found, 500 for server errors).

```
if (!isMatch)
    return res.status(401).json({ message: "Invalid credentials" });

const token = jwt.sign(
    { id: user._id, email: user.email, role },
    process.env.JWT_SECRET,
    { expiresIn: "7d" }
);
```



```

res.status(200).json({
  success: true,
  message: "Login successful",
  token,
  role: user.role,
  user: {
    name: user.name || user.full_name,
    email: user.email,
    id: user._id,
    role
  }
});

```

```

} catch (error) {
  res.status(500).json({ message: error.message });
}
};

export const getCurrentUser = async (req, res) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader)
      return res.status(401).json({ message: "No token provided" });
  }
};

```

6. Save persistent data using MongoDB and Mongoose

```

mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    server.listen(PORT, () => console.log(`Backend running on port ${PORT}`));
  })
  .catch((err) => console.error(err));

```

The screenshot shows the MongoDB Cloud Data Explorer interface. The left sidebar displays the 'Data Explorer' with a tree view of clusters. The 'employeeCluster' is expanded, showing sub-clusters like 'admin', 'employeeDB', 'departments', 'employees', 'leaves', and 'salaries'. The 'employees' cluster is selected. The main panel shows the 'Documents' tab for the 'employees' collection. A list of documents is displayed, with the first document highlighted:

```

{
  "_id": ObjectId("6918abd4e1aff5debc29d214"),
  "name": "hiren",
  "email": "hiren@gmail.com",
  "password": "$2b$10$e73k4UUMWvSHP1lrNGtiwuh36t4s04IuJ04VnZu0w1AZ1Z53cED15",
  "role": "employee"
}

```

The interface includes a search bar, a 'Generate Mock Data' button, and a 'View monitoring' button. The bottom of the screen shows a pagination bar indicating 19 documents, with the first document selected.

7. Define schemas in the /models directory.

```
backemd > models > JS EmployeeModel.js > EmployeeSchema
1  import mongoose from "mongoose";
2
3  const EmployeeSchema = new mongoose.Schema(
4    {
5      user_id: {
6        type: mongoose.Schema.Types.ObjectId,
7        ref: "User",
8        description: "Reference to User entity ID",
9      },
10     name: {
11       type: String,
12       required: true,
13       trim: true,
14       description: "Employee name",
15     },
16     email: {
17       type: String,
18       required: true,
19       unique: true,
20       match: [/^\S+@\S+\.\S+$/, "Invalid email format"],
21       description: "Employee email",
22     },
23     password: {
24       type: String,
25       required: true,
26       description: "Employee password (hashed or plain text)",
27     },
28   },
29   {
30     timestamps: true,
31   },
32 );
```

8. Use CRUD (atleast one each) operations.(above route ss)

9. Use jsonwebtoken for authentication

```
JS EmployeeModel.js M    JS verifyUser.js M X
backemd > middleware > JS verifyUser.js > verifyUser
1  import User from "../models/UserModel.js"
2  import Employee from "../models/EmployeeModel.js"
3  import jwt from "jsonwebtoken"
4  export const verifyUser = async (req, res, next) => {
5    const token = req.headers.authorization?.split(" ")[1];
6    if (!token) return res.status(401).json({ success: false, message: "No token provided" });
7    try {
8      const decoded = jwt.verify(token, process.env.JWT_SECRET);
9      let user = await User.findById(decoded.id);
10     if (!user) {
11       user = await Employee.findById(decoded.id);
12     }
13     if (!user) {
14       return res.status(404).json({ success: false, message: "User not found" });
15     }
16     req.user = user;
17     next();
18   } catch (error) {
19     console.error("Verify error:", error);
20     res.status(403).json({ success: false, message: "Invalid token" });
21   }
22 }
```

10. Use express.static() and express.json() middleware

```

app.use(morgan("dev"));
app.use(express.json());
app.use(express.static());

// ROUTES
app.use("/api/leaves", leaveRoutes);
app.use("/api/salaries", salaryRoutes);
app.use("/api/users", userRoutes);
app.use("/api/employees", empRoutes);
app.use("/api/departments", deptRoutes);

```

11. Write atleast one custom middleware function and use the middleware stack/chaining

```

JS authRoutes.js X
backend > routes > JS authRoutes.js > ...
1 // routes/authRoutes.js
2 import express from "express";
3 import {verifyUser} from "../middleware/verifyUser.js";
4
5 const router = express.Router();
6
7 router.get("/verify", verifyUser, (req, res) => {
8   res.json({
9     success: true,
10    user: req.user,
11    role: req.user.role, // this is the user decoded from the token
12  });
13 });
14
15
16 export default router;
17

```

12. Store sensitive data (API keys, database URIs) in environment variables using a .env file, and add this file to .gitignore.

